

MySQL, Tables & Joins

class 1 of 5 · 40 minutes · the real database



Dinesh Rawat

Instructor · Shorter Loop × Snapied

Today the resume project moves from a text file to a real MySQL database. We install MySQL, use MySQL Workbench to create a database and a table with our own hands, add records, and read them back. Then we cover the two ideas that make relational databases powerful: joins and normalization. Everything here is real MySQL, the same database used in production jobs. By the end you will have made a database, a table, and rows, and queried them.

1 Why a file is not enough

Our resume data has lived in `data.json`. It works in class and fails in the real world for two reasons.

Data gets lost. Two users save at the same moment. Both read the file, both write it back, and the second write erases the first user's change. No error, the data is just gone. A file cannot stop this. A database can.

Search is slow. To find one resume in a file you load all of them and loop through every one. With ten thousand records that is slow every single time. A database finds it instantly, because it was built for exactly this.

MySQL

A real database server. A separate program whose only job is to store data safely and find it fast, even with many people using it at once. It is one of the most widely used databases in the world, and what you learn here is what companies actually run.

2 Install MySQL and MySQL Workbench

Two things to install. The **MySQL Server** is the database itself. **MySQL Workbench** is a visual app to talk to it: run queries, see your tables, click around your data.

Install	What it is
MySQL Server	the database engine that stores everything. Runs in the background.
MySQL Workbench	a window into the database. You type SQL here and see results.

During install MySQL asks you to set a **root password**. Write it down. `root` is the main admin user, and you will need that password to connect from Workbench and later from your Node app. When install finishes, open Workbench and connect to `localhost` with user `root` and your password. You now have a live database in front of you.

Remember the root password. Almost every "cannot connect" problem this week is a wrong root password. If you forget it, the simplest fix is to reinstall MySQL and set a new one. Save it somewhere the moment you set it.

3 Create a database, by hand, in Workbench

A MySQL server can hold many databases, one per project. First we make ours. In Workbench, open a query tab and run this:

```
CREATE DATABASE resume_db;
```

That is it. You now have an empty database called `resume_db`. To work inside it, tell MySQL to use it:

```
USE resume_db;
```

Server vs database vs table

The server is the whole MySQL program. It holds many **databases**, one per project (`resume_db`, `shop_db`, ...). Each database holds many **tables** (`users`, `resumes`, ...). Each table holds many **rows**. Think folders inside folders: server, then database, then table, then row.

4 Create a table, by hand

Now a table to hold users. Run this in Workbench, inside `resume_db`:

```
CREATE TABLE users (
  id      INT AUTO_INCREMENT PRIMARY KEY,
  name    VARCHAR(255) NOT NULL,
  email   VARCHAR(255) NOT NULL UNIQUE
);
```

Read each line, because you are declaring the shape of your data:

Line	Means
<code>id INT AUTO_INCREMENT</code>	a whole number that MySQL fills in and increases for each row: 1, 2, 3.
<code>PRIMARY KEY</code>	this column uniquely identifies a row. one id, one exact row.
<code>VARCHAR(255)</code>	text up to 255 characters. good for names and emails.
<code>NOT NULL</code>	this column cannot be empty. every user must have a name.
<code>UNIQUE</code>	no two rows can share this value. no duplicate emails.

Refresh the schema panel on the left of Workbench and you will see `users` appear under `resume_db`, with its three columns. You built a real table.

5 Add records, and read them back

An empty table is not much. Add rows with `INSERT`:

```
INSERT INTO users (name, email) VALUES ('Himanshu', 'himanshu@example.com');
INSERT INTO users (name, email) VALUES ('Ayush', 'ayush@example.com');
INSERT INTO users (name, email) VALUES ('Tanushree', 'tanushree@example.com');
```

You did not give an `id`. MySQL filled it in: 1, 2, 3. Now read the table with `SELECT`:

```
SELECT * FROM users;
```

```
+---+-----+-----+
| id | name   | email                      |
+---+-----+-----+
| 1  | Himanshu | himanshu@example.com      |
| 2  | Ayush    | ayush@example.com         |
| 3  | Tanushree | tanushree@example.com     |
+---+-----+-----+
```

The `*` means all columns. To find one person, filter with `WHERE`:

```
SELECT * FROM users WHERE email = 'ayush@example.com';
```

`WHERE` is not optional on `UPDATE` and `DELETE`. `DELETE FROM users;` with no `WHERE` deletes every user. `DELETE FROM users WHERE id = 2;` deletes exactly one user.

6 A second table, and the foreign key

A resume belongs to a user. We make a second table and connect it to `users` with a **foreign key**, a column that holds the id of the owning user.

```
CREATE TABLE resumes (
  id      INT AUTO_INCREMENT PRIMARY KEY,
  title   VARCHAR(255) NOT NULL,
  summary TEXT,
  userId  INT,
  FOREIGN KEY (userId) REFERENCES users(id)
);

INSERT INTO resumes (title, summary, userId) VALUES ('Full Stack Intern', 'Node, Express, MySQL', 1);
INSERT INTO resumes (title, summary, userId) VALUES ('QA Intern', 'Manual + API testing', 2);
```

Foreign key

A column in one table that points to a row in another. `resumes.userId` holds a `users.id`. A resume with `userId = 1` belongs to Himanshu. The `FOREIGN KEY` constraint enforces this.

Notice `summary` is `TEXT`, not `VARCHAR`. `VARCHAR` caps at 255 characters; a real summary is longer. `TEXT` holds long text. Choosing the right type is part of database design.

7 Joins: reading two tables together

You have `users` in one table and `resumes` in another, linked by `userId`. A **join** reads them together in one query, matching each resume to its owner.

JOIN

A query that combines rows from two tables using a matching column. Here we match `resumes.userId` to `users.id`, so each resume is paired with the user it belongs to.

```
SELECT resumes.title, users.name
FROM resumes
JOIN users ON resumes.userId = users.id;
```

```
+-----+-----+
| title          | name    |
+-----+-----+
| Full Stack Intern | Himanshu |
| QA Intern       | Ayush   |
+-----+-----+
```

One query, two tables, joined on the matching id. This is the heart of a **relational** database: data lives in separate tables, and joins bring it back together on

8 Normalization: why two tables, not one

Why split users and resumes into two tables? Why not one big table with the user's name repeated on every resume row? That question is what **normalization** answers

Normalization

Organising data so each fact is stored in exactly one place. A user's name lives once, in the users table. Resumes point to it by id. You do not copy the name

Imagine the bad, un-normalized version: one table where every resume row also stores the user's name and email.

title	userName	userEmail
Full Stack Intern	Himanshu	himanshu@example.com
QA Intern	Himanshu	himanshu@example.com

Himanshu's name and email are copied on every resume he has. Two problems follow. First, **waste**: the same data stored again and again. Second, and worse, **update**

The normalized version stores the email once in users. Change it in one place and every resume, through the join, sees the new value instantly. Nothing to hunt

The rule in one line. Store each fact once, and link to it by id. Joins put the data back together when you need it. That is why relational databases use many

9 Before next class

Install MySQL and Workbench and connect as root. This is the one real setup step; get it working tonight.

Create resume_db, the users table, and add three users by hand in Workbench. Then `SELECT * FROM users` and see your rows.

Create the resumes table, add two resumes, and run the join from section 7. Seeing your own data joined across two tables is the goal of today. Next class, we

Everything here ran live in class. If a part does not click, message me the file name and the line, and read it again tonight while it is fresh.